The Journal of Open Source Software

ritest: randomization inference in Python

Tabaré Capitán[1]

1 Independent researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 22 December 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Randomization inference (RI) provides design-based inference for experiments by comparing an observed test statistic to its distribution over re-randomizations of the treatment assignment. Because the reference distribution is defined by the assignment mechanism, RI can be especially attractive when sample sizes are modest or when researchers want inference that depends primarily on the randomization protocol rather than large-sample approximations (Fisher, 1935; Imbens & Rubin, 2015).

`ritest` is a Python package implementing RI in a workflow similar to Stata's `ritest` command (Heß, 2017). It supports (i) a fast linear-model path for regression coefficients, and (ii) a generic path that accepts arbitrary user-defined statistics. For linear models, `ritest` additionally implements test inversion to obtain coefficient confidence interval (CI) bounds and a full $p(\beta_0)$ profile ("CI band") over candidate null values, designed to be practical even with large permutation counts.

Statement of need

RI is used across fields with randomized or quasi-randomized designs, including economics, political science, agricultural science, sociology, psychology, education, public health, and biosciences. Typical settings include A/B tests, laboratory experiments, greenhouse experiments, field experiments, and cluster randomized trials. In applied work, researchers often face:

- **Non-standard assignment mechanisms** (e.g., stratified randomization, cluster assignment).
- **Small or moderate samples** where asymptotic approximations may be fragile.
- **Transparent reporting goals**, such as presenting design-based p-values alongside model-based estimates.

Python has a strong ecosystem for estimation and model-based inference (e.g., linear models and generalized linear models in `statsmodels` (Seabold & Perktold, 2010) and numerical foundations in `NumPy` and `SciPy` (Harris et al., 2020; Virtanen et al., 2020)). However, an RI workflow that mirrors common applied practice—especially in econometrics and related areas—requires additional components: constrained re-randomization that matches the study design, clear Monte Carlo precision reporting for the p-value, and (for regression coefficients) a way to obtain confidence intervals by inverting the RI test. `ritest` provides these pieces in a single, documented implementation intended for integration into Python analysis pipelines.

Design and implementation

`ritest` exposes one user-facing function, `ritest()`, which takes a dataframe, a binary assignment variable, and either:

1. A linear-model specification via a formula and a focal coefficient/statistic (fast path), or

2. A user-supplied statistic function `stat_fn(df) -> float` (generic path).

Assignment mechanisms

Permutation generation supports four common assignment modes:

- **Plain**: unrestricted permutation of the assignment vector.
- **Stratified**: permutation within each stratum to preserve treated counts per stratum.
- **Cluster**: permutation of cluster-level assignments, broadcast to units within clusters.
- **Cluster-within-strata**: cluster permutation applied separately within each stratum.

These modes are designed to match typical experimental designs and to keep the RI reference distribution aligned with the original randomization protocol (Gerber & Green, 2012; Imbens & Rubin, 2015).

Fast linear-model path

For linear models, `ritest` uses a specialized OLS/WLS solver (`FastOLS`) for each permutation. Because the treatment column changes, the regression is still solved once per permutation, but permutation fits skip variance–covariance computation and avoid general-purpose model overhead. For a fixed design matrix, the focal coefficient can be written as a dot product $\hat{\beta} = c^\top y$, where c is determined by the design matrix (and, if applicable, weights). `FastOLS` computes c using Cholesky-based linear algebra.

Monte Carlo p-values and p-value confidence intervals

With R random permutations, the RI p-value is estimated by an exceedance proportion. `ritest` reports a CI for this Monte Carlo estimate by treating the exceedance count as binomial (Clopper–Pearson and normal-approximation options).

Coefficient confidence intervals by test inversion

For linear models, `ritest` can compute coefficient CI **bounds** and a full $p(\beta_0)$ **profile** over a grid of candidate nulls. Test inversion constructs a CI as the set of null values β_0 that would not be rejected by the RI test at level α.

For candidate β_0, `ritest` evaluates $p(\beta_0)$ using shifted coefficients $\beta_{\text{obs}} - \beta_0 K_{\text{obs}}$ and $\beta_r - \beta_0 K_r$ (with $K = c^\top T_{\text{metric}}$). This reuses (β_r, K_r) from the permutation fits, avoiding a refit for each β_0.

A naive implementation would refit the model for each permutation *and* for each candidate null value, which is often impractical. `ritest` implements a fast approach that reuses precomputed invariants from the observed fit and the permuted fits, making it feasible to report coefficient CIs as a default for linear models.

Examples and typical applications

`ritest` is designed to support common applications of RI:

- **Regression adjustment in randomized experiments:** RI on the treatment coefficient in an OLS model with covariates.
- **Stratified field experiments:** constrained re-randomization within pre-treatment strata.
- **Cluster randomized trials:** permutation at the cluster level, including cluster-within-strata designs.
- **Custom estimands:** any scalar statistic defined by the user, including distributional or robust summaries (generic path).

These patterns cover settings frequently encountered in applied economics and political science field experiments, psychology and education trials, and agricultural or ecological intervention studies.

Related software

Stata's `ritest` (Heß, 2017) and the R implementations (e.g., `ritest` (McDermott, 2025)) provide established RI workflows. A package providing equivalent functionality in Python does not exist. The current package also provides a significant improvement in performance relative to the implementations in Stata and R, making computation of confidence bounds and bands feasible.

Availability

`ritest` is open source, distributed on PyPI as `ritest-python` (Capitán, 2025c), and developed on GitHub (Capitán, 2025b). Documentation is hosted on the project website (Capitán, 2025a).

Capitán, T. (2025a). *Ritest documentation*. Project website. https://tabarecapitan.com/projects/ritest/

Capitán, T. (2025b). *Ritest: Randomization inference in python*. Source code repository. https://github.com/tabareCapitan/ritest

Capitán, T. (2025c). *Ritest-python*. Python package on the Python Package Index (PyPI). https://pypi.org/project/ritest-python/

Fisher, R. A. (1935). *The design of experiments*. Oliver; Boyd.

Gerber, A. S., & Green, D. P. (2012). *Field experiments: Design, analysis, and interpretation*. W. W. Norton; Company.

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Heß, S. (2017). Randomization inference with stata: A guide and software. *The Stata Journal*, *17*(3), 630–651. https://www.stata-journal.com/article.html?article=st0468

Imbens, G. W., & Rubin, D. B. (2015). *Causal inference for statistics, social, and biomedical sciences: An introduction*. Cambridge University Press. https://doi.org/10.1017/CBO9781139025751

McDermott, G. (2025). *Ritest: Randomization inference in R*. R package. https://grantmcdermott.com/ritest/

Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with python. *Proceedings of the 9th Python in Science Conference*, 92–96. https://conference.scipy.org/proceedings/scipy2010/seabold.html

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

DRAFT